

10. APIs

Módulo 5. Inteligencia Artificial

Jorge

Juvenal

Campos Ferreira

✉ juvenal.campos@tec.mx

Fuentes de los datos

Los datos que utilizamos pueden provenir de varias fuentes:

- **Datos abiertos**
- **Información de la web**
- **APIs**

Librerías a descargar

- * httr
- * jsonlite
- * inegiR
- * tidyquant
- * tuber

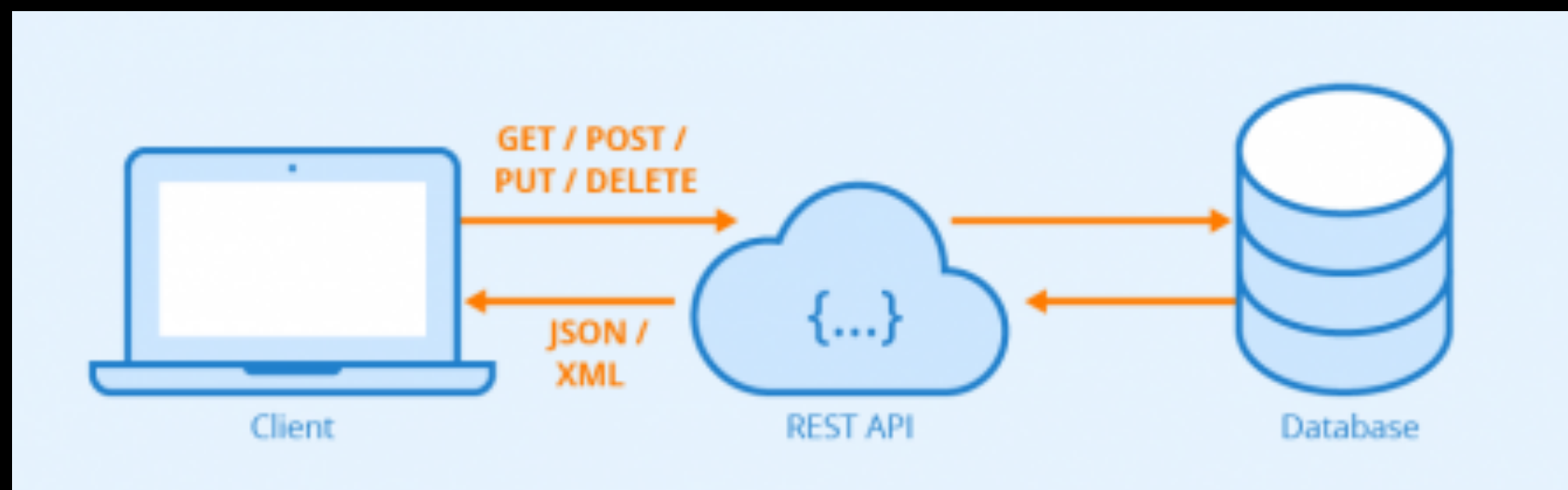


Descargar también **Ollama** y los modelos **Qwen** y **Llama**

APIs

API (Application Programming Interface o Interfaz de Programación de Aplicaciones).

Las APIs son **componentes** del servidor que hacen que sea fácil que nuestro código interactúe con un servicio y obtenga datos de este.



APIs



APIs en R



En una contexto de R:

- Las APIs **son como páginas web, pero para máquinas.**
- Pueden ser utilizadas para **importar grandes cantidades de datos** de manera automática.
- Nos permiten hacer **búsquedas para pedazos específicos de los datos**, dentro de un universo de datos muy grande.

APIs en R



– Para R, existen dos formas de usar las APIs.

Con la librería `{httr}`

La librería `httr` nos permite hacer consultas a las APIs de una manera tradicional (muy similar a como lo haría un programador).

A través de API Clients

Son librerías de R específicas para acceder a las APIs de diferentes servicios. Ejemplos: `{spotifyr}`, `{rtweet}`, `{tuber}`, o `{pageviews}`.

Cientes APIs para R

La forma fácil

- Siempre usar un cliente, cuando sea posible.
- Son interfaces nativas para acceder a las APIs con código y funciones de R. (Son como cualquier otra librería).
- Ocultan completamente la API, y nos permite acceder solo a los resultados estructurados.
- Nos permiten leer los datos a través de objetos de R, en vez de tener que lidiar con archivos JSON.

Etiqueta al usar APIs



– Muchas APIs nos exigen un registro previo, esto con el fin de **controlar el acceso y el uso moderado.**

– Para usar una API, tras el registro, se nos dan tokens de acceso para controlar el acceso y el consumo de los servicios. **Estas llaves muchas veces van incluidas en nuestro código.**

Etiqueta al usar APIs



- Sobrecargar una API con solicitudes puede **reducir la calidad del servicio** tanto para ti como para otros usuarios.
- Muchas APIs regulan el consumo de datos: **permitiendo cierto numero de consultas durante cierto tiempo.**

¿Y si no hay cliente de APIs?



Cuando no hay un paquete intermediario que nos permita comunicarnos con un API, hay que **conectarnos de manera directa con este servicio.**

Para esto, hay que utilizar las librerías *{httr}* y *{jsonlite}*

Llamadas http

GET and POST request in theory

HTTP requests

- Conversation between your machine and the server
- First: what you want to happen
- 'methods' - different requests for different tasks

GET and POST

- GET: 'get me something'
- POST: 'have something of mine'



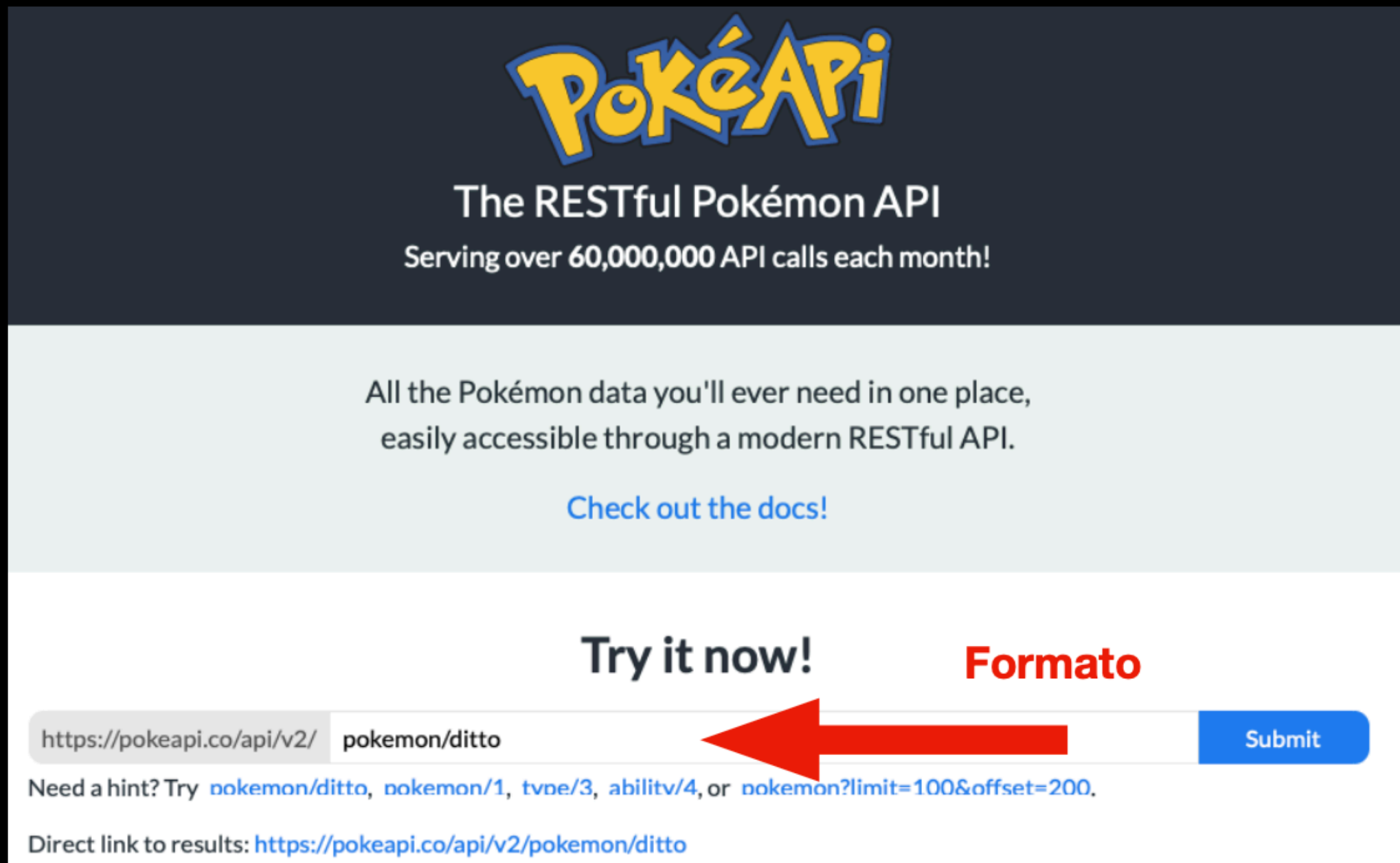
El bueno

Other tyoes

- HEAD - just like `head()`
- DELETE - 'remove this thing'
- Many others! But GET and POST are the big ones!

Haciendo una conexión directa

Paso 0. Construyo mi petición en el formato que me lo indique la documentación.



The screenshot shows the PokéAPI website. At the top, the logo "PokéAPI" is displayed in a stylized yellow font with a blue outline. Below it, the text "The RESTful Pokémon API" and "Serving over 60,000,000 API calls each month!" are shown. A light blue section contains the text "All the Pokémon data you'll ever need in one place, easily accessible through a modern RESTful API." and a link "Check out the docs!". The bottom section features a "Try it now!" button and a "Formato" label. Below these is a search bar with the URL "https://pokeapi.co/api/v2/pokemon/ditto" and a "Submit" button. A large red arrow points from the "Formato" label to the search bar. Below the search bar, there is a hint: "Need a hint? Try [pokemon/ditto](#), [pokemon/1](#), [type/3](#), [ability/4](#), or [pokemon?limit=100&offset=200](#)." and a direct link to results: "Direct link to results: <https://pokeapi.co/api/v2/pokemon/ditto>".

PokéAPI
The RESTful Pokémon API
Serving over 60,000,000 API calls each month!

All the Pokémon data you'll ever need in one place,
easily accessible through a modern RESTful API.

[Check out the docs!](#)

Try it now! **Formato**

Need a hint? Try [pokemon/ditto](#), [pokemon/1](#), [type/3](#), [ability/4](#), or [pokemon?limit=100&offset=200](#).

Direct link to results: <https://pokeapi.co/api/v2/pokemon/ditto>

Haciendo una conexión directa



Paso 1. Ya que vi como se tenía que construir la pregunta (llamada), la construyo.

parte_constante + parte_variable

```
# Llamada a la API  
call1 <- paste0("https://pokeapi.co/api/v2/",  
               "pokemon/pikachu/")
```

Haciendo una conexión directa



Paso 2. Dada la dirección de la API (la llamada), hago una GET request, con la función *httr::GET()*, lo cual nos devuelve una respuesta de la información solicitada.

```
# Realizamos la llamada (Obtenemos la respuesta)
llamada <- GET(call1)
llamada # Status 200: Todo ok.
class(llamada) # Tipo Response
```

Objeto response



```
> llamada
Response [https://pokeapi.co/api/v2/pokemon/pikachu/]
Date: 2020-10-01 18:35
Status: 200
Content-Type: application/json; charset=utf-8
Size: 191 kB
```

Haciendo una conexión directa



Paso 3. De mi objeto response, aplico la función *httr::content()* para acceder al contenido.

```
# De la llamada, obtenemos la respuesta en formato JSON
```

```
# Formato JSON: https://en.wikipedia.org/wiki/JSON
```

```
get_data <- content(llamada, "text")
```

```
get_data
```

```
class(get_data)
```

Datos tipo JSON



```
\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\", \"version_group\": {\"name\": \"crystal\", \"url\": \"https://pokeapi.co/api/v2/version-group/4/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"tutor\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/3/\"}, \"version_group\": {\"name\": \"emerald\", \"url\": \"https://pokeapi.co/api/v2/version-group/6/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\"}, \"version_group\": {\"name\": \"diamond-pearl\", \"url\": \"https://pokeapi.co/api/v2/version-group/8/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\"}, \"version_group\": {\"name\": \"platinum\", \"url\": \"https://pokeapi.co/api/v2/version-group/9/\"}}, {\"level_learned_at\": 0, \"move_learn_method\": {\"name\": \"machine\", \"url\": \"https://pokeapi.co/api/v2/move-learn-method/4/\"}, \"version_group\": {\"name\": \"heartgold-soulsilver\", \"url\": \"https://pokeapi.co/api/v2/version-group/10/\"}}] [\"move\": {\"name\"
```

Haciendo una conexión directa



Paso 4. Una vez que tengo mi objeto respuesta arrojó el resultado en formato *JSON*, lo convierto en un objeto tipo *list* con la función *jsonlite::fromJSON()*

```
# Del JSON, convertimos este texto en un objeto lista
get_data_JSON <- fromJSON(get_data, flatten = TRUE)
class(get_data_JSON)
```

Objeto ordenado en una lista

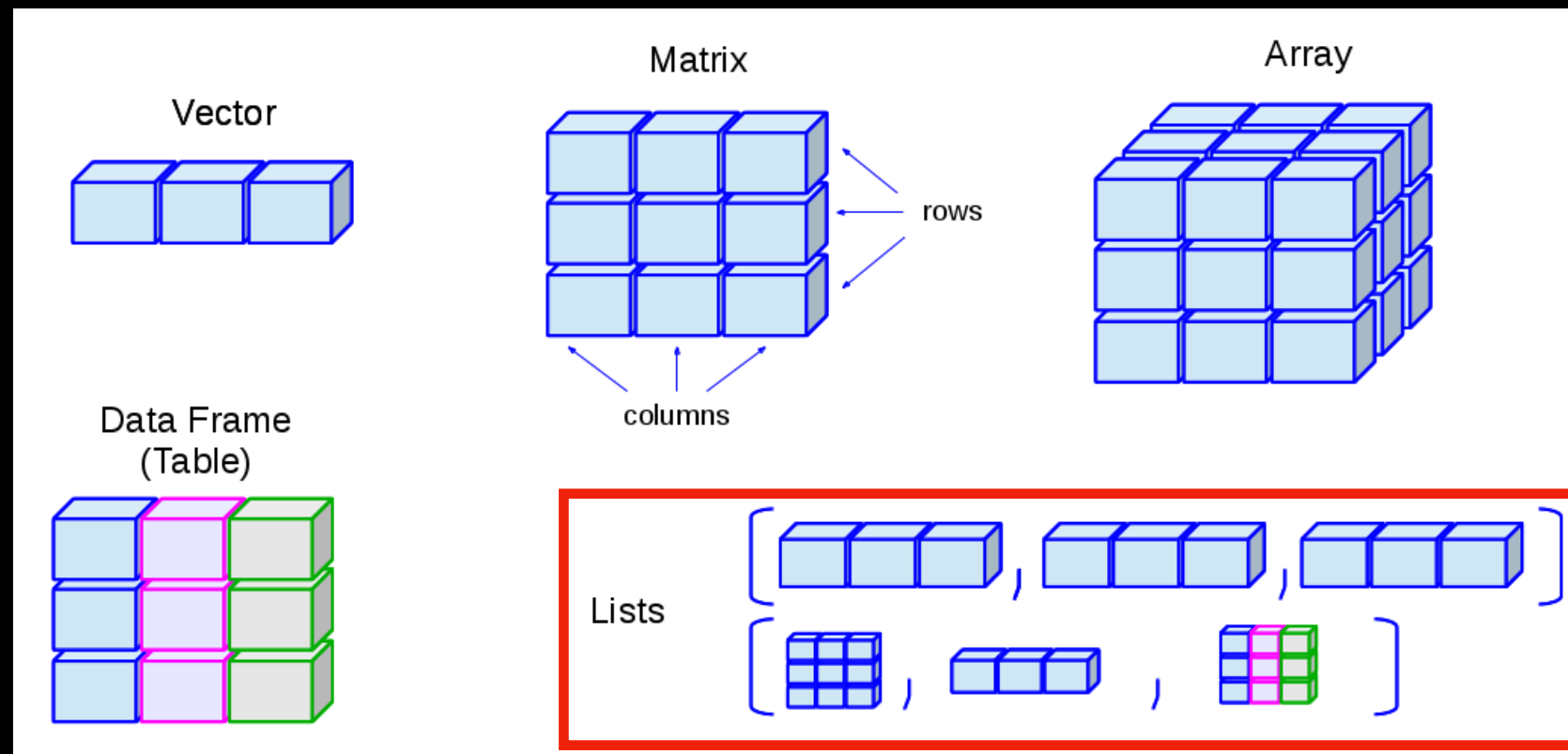


```
• get_data_JSON List of 17
  abilities : 'data.frame': 2 obs. of 4 variables:
    ..$ is_hidden : logi [1:2] FALSE TRUE
    ..$ slot : int [1:2] 1 3
    ..$ ability.name: chr [1:2] "static" "lightning-r...
    ..$ ability.url : chr [1:2] "https://pokeapi.co/a...
  base_experience : int 112
  forms : 'data.frame': 1 obs. of 2 variables:
    ..$ name: chr "pikachu"
    ..$ url : chr "https://pokeapi.co/api/v2/pokemon-...
  game_indices : 'data.frame': 20 obs. of 3 variable...
    ..$ game_index : int [1:20] 84 84 84 25 25 25 25 ...
    ..$ version.name: chr [1:20] "red" "blue" "yellow...
    ..$ version.url : chr [1:20] "https://pokeapi.co/...
  height : int 4
  held_items : 'data.frame': 2 obs. of 3 variables:
    ..$ version_details: List of 2
    .. ..$ : 'data.frame': 10 obs. of 3 variables:
```

Cosas a tomar en cuenta de las APIs

- * **A veces, las API cuestan**, ya que son parte del negocio de las empresas.
- * El uso de muchas APIs implica registrarse a sus servicios, para obtener un token de acceso.
- * Las APIs tienen guías de uso y documentación que hay que leer para usarlas
- * Se recomienda guardar los tokens con la funcion `renviron::use_this()`

Listas



Una lista es como un vector, en donde el contenido de cada casilla puede ser de **cualquier** tipo (otra lista adentro, un vector, una tibble o dataframe, etc.).

Formato JSON



```
{
  "users": [
    {
      "userId": 1,
      "firstName": "Chris",
      "lastName": "Lee",
      "phoneNumber": "555-555-5555",
      "emailAddress": "clee@fileinfo.com"
    },
    {
      "userId": 2,
      "firstName": "Action",
      "lastName": "Jackson",
      "phoneNumber": "555-555-5556",
      "emailAddress": "ajackson@fileinfo.com"
    }
  ],
}
```

```
"store": {
  "book": [
    {
      "category": "reference",
      "author": "Nigel Rees",
      "title": "Sayings of the Century",
      "price": 8.95
    },
    {
      "category": "fiction",
      "author": "Evelyn Waugh",
      "title": "Sword of Honour",
      "price": 12.99
    },
    {
      "category": "fiction",
      "author": "Herman Melville",
      "title": "Moby Dick",
      "isbn": "0-553-21311-3",
      "price": 8.99
    }
  ],
}
```

Ejemplos:

https://github.com/JuveCampos/Shapes_Resiliencia_CDMX_CIDE/raw/master/Infraestructura/Pozos_Sacmex2.geojson;
Ver archivo carpeta local.

Formato JSON



Un archivo *.json (JavaScript Object Notation) es un **archivo que almacena estructuras simples** de datos.

- Este formato es un **estándar para el intercambio de información**, y se usa para transmitir información entre una aplicación y un servidor.
- Estos archivos son **ligeros, basados en texto, legibles por humanos** y pueden ser editados por un editor de texto.
- Es el **formato** que se utiliza para **dar información proveniente de un API**.
- Para leerlos **en R**, tenemos que usar funciones de la librería **{jsonlite}**.

```
{
  "tag": "select",
  "name": "ctl00$plnMain$ddlClasses",
  "id": "plnMain_ddlClasses",
  "class": "sg-combobox",
  "children": [
    {
      "tag": "option",
      "selected": "selected",
      "value": "ALL",
      "html": "(All Classes)"
    },
    {
      "tag": "option",
      "value": "1319202|1",
      "html": "0020A - 36   STUDY HALL (INSTRUCT)"
    },
    {
      "tag": "option",
      "value": "1319203|1",
      "html": "0020B - 33   STUDY HALL (INSTRUCT)"
    },
    {
      "tag": "option",
      "value": "1232138|1",
      "html": "0113A - 1   APENGLAN A"
    },
    {
      "tag": "option",
      "value": "1232137|1",
      "html": "0113B - 1   APENGLAN B"
    },
    {
      "tag": "option",
      "value": "1229605|1",
      "html": "0284A - 2   PRE CALC PREAP A"
    },
    {
      "tag": "option",
      "value": "1229608|1",
      "html": "0284B - 2   PRE CALC PREAP B"
    },
    {
      "tag": "option",
      "value": "1229088|1",

```

Ejercicio práctico

- * Utilice la PokeAPI y genere una tabla con el peso y la altura de los primeros 151 Pokemon, incluyendo liga a sus sprites de los juegos.
- * Utilice el API de Open Meteo API y obtenga el clima pronosticado para el día de hoy para las coordenadas del Tec de Monterrey Campus Santa Fe
- * Utilice el API de Tidyquant para obtener el precio de las acciones de Apple.
- * Utilice el API del INEGI para determinar que entidad de la república tiene el mayor PIB estatal
- * Consiga sus credenciales de las API de Youtube y de Gemini (o si quiere pagar, de OpenAI o de Claude).
- * Con la API de Youtube y el paquete `{tuber}`, descargue los comentarios de alguna noticia relacionada con los nuevos impuestos a videojuegos violentos
- * Con la API de Gemini y el paquete `{ellmer}`, determine si los mensajes de arriba son discurso de odio
- * Con Ollama, genere una consulta interna para realizar el mismo proceso.

Variables de ambiente

Son pares clave-valor que el sistema operativo o que R utilizan para almacenar información de configuración.

Funcionan como un contexto que afecta a la ejecución de tu sesión en R, sin necesidad de estar definidas en el script.

Variables de ambiente

Una variable normal existe solo dentro de tu entorno de trabajo.

Una variable de ambiente vive en el sistema y es leída por R. Puede afectar como funcionan los paquetes sin que tengas que ponerlas explícitamente en el código.

Suelen usarse típicamente para guardar llaves de acceso a APIs. Tiene la ventaja de la **seguridad** (evitas poner tus contraseñas en el código) y de **mantenibilidad** (cambias solo una contraseña en vez de cambiar la llave en cada código).

Variables de ambiente

Ejemplo de cómo se vería el archivo .Renviron que carga las variables de ambiente.

Para modificar el archivo .Renviron utilizamos la función `usethis::edit_r_environ()`

```
# Ejemplo en un archivo .Renviron o en el sistema
R_LIBS_USER=/Users/juvenal/R/Library
DB_HOST=database.myserver.com
DB_USER=analyst_user
DB_PASS=SuperSecreta123
API_KEY_YOUTUBE=AIZAaSyAxxxxxxx-xxxxx-xxxx
API_KEY_SPOTIFY=Sp0tifyK3y_123456789
LANG=es_MX.UTF-8
PATH=/usr/local/bin:/usr/bin:/bin:/usr/sbin:/sbin
```

lapply()

Es una función que **nos permite hacer bucles sobre listas**. Tiene la ventaja que no nos exige generar tablas vacías al inicio de un bucle (como lo hacíamos con *for*). Nos devuelve como resultado también una lista.

```
lapply(X, FUN, ...)
```

- **X** : un objeto tipo lista o vector.
- **FUN** : la función que se aplicará a cada elemento.
- **...** : argumentos adicionales que quieras pasarle a la función.

lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

X: puede ser 1:100 (del uno al 100), puede ser los nombres de los archivos, lo que sea sobre lo que querramos iterar.

FUN: La función que le queremos aplicar a los valores de **X**. Los argumentos adicionales se le pueden aplicar con ...

lapply()

```
lapply(X, FUN, ...)
```

- **X**: un objeto tipo lista o vector.
- **FUN**: la función que se aplicará a cada elemento.
- **...**: argumentos adicionales que quieras pasarle a la función.

Más sobre el FUN:

Al igual que en python, en un lapply podemos aplicar funciones anonimas (lambdas) que se van a ejecutar a todos los elementos de **X**.

lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

lapply()

Ejemplo:

```
palabras <- list("hola", "mundo", "INEGI")  
lapply(palabras, function(x) toupper(x))
```

Función anónima

Salida:

```
[[1]]  
[1] "HOLA"  
  
[[2]]  
[1] "MUNDO"  
  
[[3]]  
[1] "INEGI"
```

lapply()

Si la lista-resultado de un `lapply()` tiene las mismas dimensiones en todos sus elementos, con la función `do.call(rbind, .)` estas listas se pueden convertir en tablas.

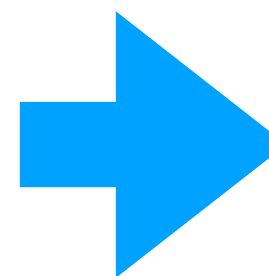
```
library(tibble)

# Tenemos un vector con nombres
personas <- c("Ana", "Luis", "Marta")

# Usamos lapply() para generar una lista de listas
mi_lista <- lapply(seq_along(personas), function(i) {
  tibble(
    nombre = personas[i],
    edad   = 20 + i * 5,
    ciudad = c("CDMX", "Monterrey", "Guadalajara")[i]
  )
})

# Convertimos la lista en tibble con do.call()
mi_tibble <- mi_lista %>% do.call(rbind, .)

mi_tibble
```



nombre	edad	ciudad
<chr>	<dbl>	<chr>
Ana	25	CDMX
Luis	30	Monterrey
Marta	35	Guadalajara